UNIVERSIDAD DISTRITAL FRANCISCO JOSE DE CALDAS

Curso de Bases de Datos — Proyecto Final

CineClub Vintage

Sistema de Alquiler de Películas

Diagrama Entidad-Relación, Modelo Relacional, Diccionario de Datos, Vistas, Triggers y Guía de Sustentación

Estudiante	Isabella Andrea Ramon Vanegas
Código	20251020162
Motor de base de datos	PostgreSQL 18-17(compatible con 14 o superior)
Base de datos	cineclub_vintage — 6 tablas en 3FN, 3 vistas, 5 triggers (3 obligatorios + 2 auxiliares)
Fecha	10 de noviembre de 2026

Contenido

1. Diagrama Entidad-Relación — Notación Crow's Foot 3
2. Modelo Relacional 4
3. Diccionario de Datos 4
3.1 Tabla PELICULA 4
3.2 Tabla COPIA 4
3.3 Tabla CLIENTE 5
3.4 Tabla EMPLEADO 5
3.5 Tabla ALQUILER 5
3.6 Tabla DETALLE_ALQUILER 6
4. Relaciones y Cardinalidades 7
5. Justificación de la Tercera Forma Normal 7
5.1 Atributos derivados exigidos por el enunciado 7
6. Vistas de Negocio y Lógica Activa 9
6.1 Vistas 9
6.2 Triggers 9
7. Resultados del Script de Validación 10
8. Guía de Sustentación: Concurrencia, Aislamiento y Bloqueos 11
8.1 Conceptos 11
8.2 Por qué el diseño evita el doble alquiler 11
8.3 Guion de la demostración (script 03) 11
9. Archivos Entregados y Orden de Ejecución 12

(Si la tabla de contenido aparece vacía, haga clic derecho sobre ella en Word y elija "Actualizar campos".)

1. Diagrama Entidad-Relación — Notación Crow's Foot

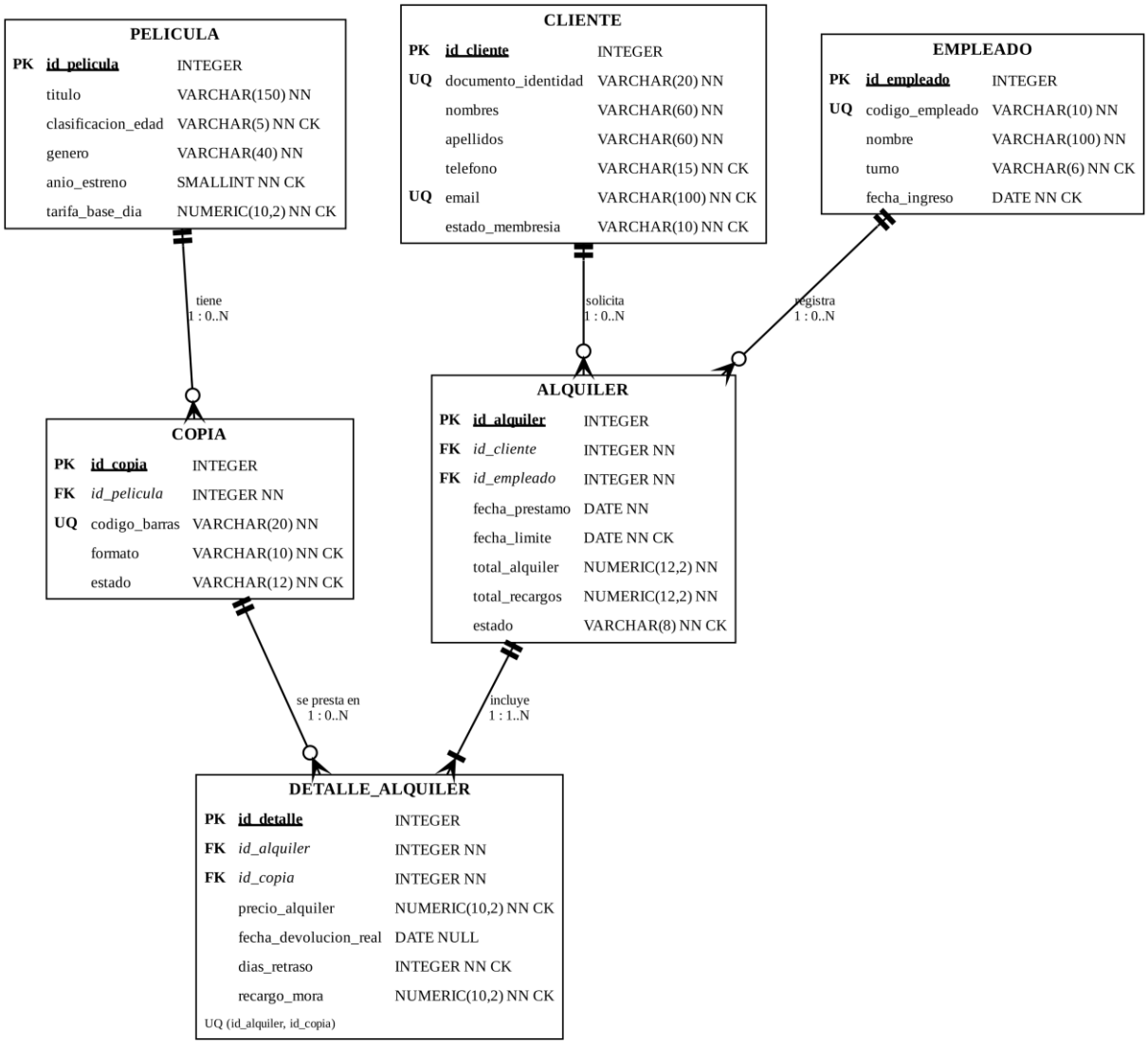


Figura 1. Diagrama ER del sistema CineClub Vintage (6 entidades, 5 relaciones).

PK = llave primaria (subrayada)	FK = llave foránea (cursiva)	UQ = UNIQUE	NN = NOT NULL	CK = CHECK	┼┼ exactamente uno · ○< cero o muchos · ┼< uno o muchos

2. Modelo Relacional

Notación: subrayado = llave primaria; *cursiva* = llave foránea (→ tabla referenciada).

- **PELICULA** (id_pelicula, titulo, clasificacion_edad, genero, anio_estreno, tarifa_base_dia)
- **COPIA** (id_copia, *id_pelicula* → PELICULA, codigo_barras, formato, estado)
- **CLIENTE** (id_cliente, documento_identidad, nombres, apellidos, telefono, email, estado_membresia)
- **EMPLEADO** (id_empleado, codigo_empleado, nombre, turno, fecha_ingreso)
- **ALQUILER** (id_alquiler, *id_cliente* → CLIENTE, *id_empleado* → EMPLEADO, fecha_prestamo, fecha_limite, total_alquiler, total_recargos, estado)
- **DETALLE_ALQUILER** (id_detalle, *id_alquiler* → ALQUILER, *id_copia* → COPIA, precio_alquiler, fecha_devolucion_real, dias_retraso, recargo_mora)

Las llaves primarias son sustitutas (INTEGER GENERATED BY DEFAULT AS IDENTITY). Las llaves naturales (código de barras, documento de identidad, código de empleado y correo electrónico) se protegen con restricciones UNIQUE.

Nota: el enunciado nombra el atributo *año_estreno*; en el script se usa *anio_estreno* para evitar la letra ñ en identificadores SQL y garantizar la portabilidad entre clientes y codificaciones.

3. Diccionario de Datos

3.1 Tabla PELICULA

Columna	Tipo	Nulo	Restricción	Descripción
id_pelicula	INTEGER	No	PK, IDENTITY	Identificador de la película
titulo	VARCHAR(150)	No	UQ (titulo, anio_estreno)	Título comercial
clasificacion_edad	VARCHAR(5)	No	CK ∈ {G, PG, PG-13, R, NC-17}	Clasificación por edad
genero	VARCHAR(40)	No	—	Género cinematográfico
anio_estreno	SMALLINT	No	CK entre 1888 y 2100	Año de estreno
tarifa_base_dia	NUMERIC(10,2)	No	CK > 0	Valor por día de alquiler (COP)

3.2 Tabla COPIA

Columna	Tipo	Nulo	Restricción	Descripción
id_copia	INTEGER	No	PK, IDENTITY	Identificador del disco físico

Columna	Tipo	Nulo	Restricción	Descripción
id_pelicula	INTEGER	No	FK → PELICULA (RESTRICT)	Película a la que pertenece
codigo_barras	VARCHAR(20)	No	UNIQUE	Etiqueta física del disco
formato	VARCHAR(10)	No	CK ∈ {DVD, Blu-ray, 4K}	Formato del soporte
estado	VARCHAR(12)	No	CK ∈ {Disponible, Alquilada, Dañada}; DEFAULT Disponible	Lo mantienen los triggers

3.3 Tabla CLIENTE

Columna	Tipo	Nulo	Restricción	Descripción
id_cliente	INTEGER	No	PK, IDENTITY	Identificador del socio
documento_identidad	VARCHAR(20)	No	UNIQUE	Cédula u otro documento
nombres	VARCHAR(60)	No	—	Nombres del socio
apellidos	VARCHAR(60)	No	—	Apellidos del socio
telefono	VARCHAR(15)	No	CK dígitos, 7 a 15	Contacto para cobro de mora
email	VARCHAR(100)	No	UNIQUE; CK formato de correo	Correo electrónico
estado_membresia	VARCHAR(10)	No	CK ∈ {Activa, Suspendida}; DEFAULT Activa	Habilita o bloquea alquileres

3.4 Tabla EMPLEADO

Columna	Tipo	Nulo	Restricción	Descripción
id_empleado	INTEGER	No	PK, IDENTITY	Identificador del empleado
codigo_empleado	VARCHAR(10)	No	UNIQUE	Código interno (EMP-001)
nombre	VARCHAR(100)	No	—	Nombre completo
turno	VARCHAR(6)	No	CK ∈ {Mañana, Tarde, Noche}	Jornada laboral
fecha_ingreso	DATE	No	CK ≥ 1990-01-01; DEFAULT hoy	Fecha de vinculación

3.5 Tabla ALQUILER

Columna	Tipo	Nulo	Restricción	Descripción
id_alquiler	INTEGER	No	PK, IDENTITY	Número de la transacción
id_cliente	INTEGER	No	FK → CLIENTE (RESTRICT)	Quién alquila
id_empleado	INTEGER	No	FK → EMPLEADO (RESTRICT)	Quién registra
fecha_prestamo	DATE	No	DEFAULT CURRENT_DATE	Fecha de entrega
fecha_limite	DATE	No	CK fecha_limite > fecha_prestamo	Fecha pactada de devolución

Columna	Tipo	Nulo	Restricción	Descripción
total_alquiler	NUMERIC(12,2)	No	CK ≥ 0; DEFAULT 0	Σ precio_alquiler (trigger)
total_recargos	NUMERIC(12,2)	No	CK ≥ 0; DEFAULT 0	Σ recargo_mora (trigger)
estado	VARCHAR(8)	No	CK ∈ {Abierto, Cerrado}; DEFAULT Abierto	Se cierra al devolver todo (trigger)

3.6 Tabla DETALLE_ALQUILER

Columna	Tipo	Nulo	Restricción	Descripción
id_detalle	INTEGER	No	PK, IDENTITY	Línea del alquiler
id_alquiler	INTEGER	No	FK → ALQUILER (CASCADE)	Cabecera a la que pertenece
id_copia	INTEGER	No	FK → COPIA (RESTRICT); UQ (id_alquiler, id_copia)	Disco prestado
precio_alquiler	NUMERIC(10,2)	No	CK ≥ 0	Tarifa × días pactados (trigger; valor histórico)
fecha_devolucion_real	DATE	Sí	Validada por trigger	NULL mientras no se devuelva
dias_retraso	INTEGER	No	CK ≥ 0; DEFAULT 0	Calculado por trigger
recargo_mora	NUMERIC(10,2)	No	CK ≥ 0; DEFAULT 0	Calculado por trigger

4. Relaciones y Cardinalidades

Relación	Padre	Hijo	Lectura	FK / acción
PELICULA – COPIA	1	0..N	Una película tiene cero o muchas copias; cada copia pertenece a una sola película.	copia.id_pelicula · RESTRICT
CLIENTE – ALQUILER	1	0..N	Un cliente realiza cero o muchos alquileres; cada alquiler es de un único cliente.	alquiler.id_cliente · RESTRICT
EMPLEADO – ALQUILER	1	0..N	Un empleado registra cero o muchos alquileres; cada alquiler lo registra un empleado.	alquiler.id_empleado · RESTRICT
ALQUILER – DETALLE	1	1..N	Un alquiler incluye una o más copias; cada línea pertenece a un alquiler.	detalle.id_alquiler · CASCADE
COPIA – DETALLE	1	0..N	Una copia aparece en cero o muchos préstamos a lo largo del tiempo; cada línea presta una copia.	detalle.id_copia · RESTRICT

La relación muchos a muchos entre ALQUILER y COPIA se resuelve con la entidad asociativa DETALLE_ALQUILER, que además almacena los atributos propios de cada préstamo (precio pactado, fecha de devolución y mora). La restricción UNIQUE (id_alquiler, id_copia) impide repetir la misma copia dentro de un alquiler. El mínimo «1» del lado de DETALLE es una regla conceptual: el flujo operativo crea la cabecera y luego sus líneas dentro de la misma transacción.

5. Justificación de la Tercera Forma Normal

- **Primera Forma Normal (1FN).** Todos los atributos son atómicos (nombres y apellidos separados, un teléfono y un correo por cliente) y no existen grupos repetitivos: las varias copias de un alquiler se registran como filas de DETALLE_ALQUILER y no como columnas.
- **Segunda Forma Normal (2FN).** Todas las llaves primarias son simples (una sola columna), por lo que no pueden existir dependencias parciales.
- **Tercera Forma Normal (3FN).** Ningún atributo no clave depende de otro atributo no clave: los datos de la película (título, género, tarifa) residen sólo en PELICULA y la copia los obtiene por llave foránea; los datos del cliente y del empleado no se repiten en ALQUILER; y la información de la copia no se repite en DETALLE_ALQUILER.

5.1 Atributos derivados exigidos por el enunciado

El enunciado exige almacenar *total_alquiler*, *total_recargos*, *dias_retraso* y *recargo_mora*. Son valores calculables, pero se conservan por razones de negocio (desnormalización controlada) y su consistencia está garantizada porque **sólo los escriben los triggers**: si alguien intenta modificarlos manualmente, el trigger los recalcula (caso C4 del script de validación). El atributo *precio_alquiler*

no es redundante: es el precio histórico pactado, que debe conservarse aunque la tarifa de la película cambie posteriormente.

6. Vistas de Negocio y Lógica Activa

6.1 Vistas

Vista	Qué muestra	Lógica principal
V_CatalogoDisponible	Título, género, clasificación, formato y número de copias disponibles.	JOIN película–copia, WHERE estado = 'Disponible', GROUP BY película y formato.
V_AlquileresEnMora	Alquileres abiertos y vencidos: cliente, documento, teléfono, correo, título, código de barras, días de mora, recargo estimado y si el cliente ya está bloqueado (más de 3 días).	estado = 'Abierto' AND fecha_limite < CURRENT_DATE AND fecha_devolucion_real IS NULL; días = CURRENT_DATE − fecha_limite.
V_ReporteRecaudacionMensual	Por año y mes: cantidad de alquileres, tarifas base facturadas, multas recaudadas, total y porcentaje de multas.	Tarifas por mes del préstamo y multas por mes de la devolución real (UNION ALL), agregadas con GROUP BY.

6.2 Triggers

Trigger	Momento / tabla	Comportamiento
TR_ValidarClienteHabilitado (obligatorio)	BEFORE INSERT · ALQUILER	Rechaza el alquiler si la membresía está 'Suspendida' o si el cliente tiene copias sin devolver con más de 3 días de atraso. Usa SELECT … FOR SHARE sobre el cliente y fuerza estado 'Abierto' con totales en 0.
TR_PrepararDetalleAlquiler (auxiliar)	BEFORE INSERT · DETALLE_ALQUILER	Calcula precio_alquiler = tarifa_base_dia × (fecha_limite − fecha_prestamo); impide agregar líneas a un alquiler cerrado o insertar una línea ya devuelta.
TR_ActualizarEstadoCopiaAlquiler (obligatorio)	AFTER INSERT · DETALLE_ALQUILER	UPDATE copia SET estado = 'Alquilada' WHERE id_copia = … AND estado = 'Disponible'. Si no afecta filas lanza una excepción descriptiva y se revierte toda la sentencia.
TR_CalcularMoraYDisponibilidad (obligatorio)	BEFORE UPDATE · DETALLE_ALQUILER	Al registrar fecha_devolucion_real valida la fecha (no anterior al préstamo ni futura), calcula dias_retraso = máx(0, devolución − límite) y recargo_mora = días × tarifa × 1,5, y devuelve la copia a 'Disponible'. Impide anular una devolución.
TR_SincronizarTotalesAlquiler (auxiliar)	AFTER INSERT / UPDATE / DELETE · DETALLE_ALQUILER	Recalcula total_alquiler y total_recargos, y cierra el alquiler cuando todas sus copias han sido devueltas.

Regla de mora adoptada: cada día de retraso cuesta el 150 % de la tarifa diaria de la película. Ejemplo: *Casablanca* (3 500 COP por día) devuelta 7 días tarde → 7 × 3 500 × 1,5 = 36 750 COP.

7. Resultados del Script de Validación

El script 02_CineClub_Validacion_Triggers.sql ejecuta 24 casos dentro de una transacción que termina en ROLLBACK, de modo que los datos de prueba no se alteran. Resultado obtenido en PostgreSQL 16: **24 de 24 casos PASAN**.

Caso	Prueba	Esperado	Obtenido
A1	Cliente activo sin mora abre alquiler	Aceptado	Aceptado
A2	Cliente con membresía Suspendida	Rechazado	Rechazado
A3	Cliente suspendido tras mora de 10 días	Rechazado	Rechazado
A4	Cliente con préstamo vencido hace 7 días	Rechazado	Rechazado
A5	Caso límite: exactamente 3 días de atraso	Aceptado	Aceptado
B1	Alquilar copia Disponible (pasa a Alquilada)	Aceptado	Aceptado
B2	Alquilar una copia ya Alquilada	Rechazado	Rechazado
B3	Alquilar una copia Dañada	Rechazado	Rechazado
B4	Lote con una copia válida y una no disponible (atomicidad)	Rechazado, sin cambios	Rechazado, sin cambios
B5	Agregar copia a un alquiler Cerrado	Rechazado	Rechazado
C1	Devolución anticipada	0 días; copia Disponible	0 días; copia Disponible
C2	Devolución 7 días tarde (2 copias)	Recargo 68 250	Recargo 68 250
C3	Cliente moroso alquila tras ponerse al día	Aceptado	Aceptado
C4	Poner recargo_mora = 0 manualmente	Se recalcula	Se recalcula
C5–C7	Devolución anterior al préstamo / futura / anulación de devolución	Rechazado	Rechazado
D1–D7	UNIQUE, CHECK (formato, correo, fechas, tarifa), FK y RESTRICT	Rechazado	Rechazado

8. Guía de Sustentación: Concurrencia, Aislamiento y Bloqueos

8.1 Conceptos

- **Transacción y ACID.** Un alquiler (cabecera, líneas y cambio de estado de las copias) es una unidad atómica: se aplica todo o nada. Las excepciones de los triggers abortan la transacción y PostgreSQL deshace todos sus cambios.
- **MVCC.** PostgreSQL mantiene versiones de las filas: los lectores no bloquean a los escritores ni viceversa. Por eso consultar V_CatalogoDisponible nunca queda en espera aunque otra caja esté alquilando.
- **Bloqueos de fila.** UPDATE, DELETE y SELECT … FOR UPDATE toman un bloqueo exclusivo sobre la fila hasta el COMMIT o ROLLBACK; una segunda transacción que quiera modificar la misma fila debe esperar. FOR SHARE (usado sobre el cliente) permite otros lectores, pero impide que cambien la membresía mientras se valida.
- **Niveles de aislamiento.** READ COMMITTED (predeterminado): cada sentencia ve lo confirmado al iniciar y, si espera un bloqueo, re-evalúa el WHERE sobre la versión nueva. REPEATABLE READ: toda la transacción ve una única instantánea; si intenta modificar una fila cambiada por otra transacción confirmada, falla con SQLSTATE 40001 y debe reintentarse. SERIALIZABLE: además detecta anomalías de escritura sesgada (write skew). PostgreSQL no permite lecturas sucias en ningún nivel.
- **Interbloqueo (deadlock).** Si A espera a B y B espera a A, PostgreSQL lo detecta tras deadlock_timeout (1 s) y aborta una de las dos transacciones. Se previene accediendo a las tablas siempre en el mismo orden.

8.2 Por qué el diseño evita el doble alquiler

El trigger no hace «consultar y luego actualizar», lo que crearía una condición de carrera, sino una única sentencia atómica: UPDATE copia SET estado = 'Alquilada' WHERE id_copia = … AND estado = 'Disponible'. Si dos cajas alquilan la misma copia a la vez, la segunda queda bloqueada en esa fila; cuando la primera confirma, READ COMMITTED vuelve a evaluar la condición, ya no encuentra la copia disponible y el trigger lanza la excepción.

8.3 Guion de la demostración (script 03)

Escenario	Pasos (3 ventanas psql)	Resultado verificado en PostgreSQL 16
1. Misma copia en dos cajas (READ COMMITTED)	A: BEGIN y alquila la copia 26 sin COMMIT. B: BEGIN e intenta la copia 26, queda en espera. C: consulta pg_stat_activity y pg_locks. A: COMMIT.	C muestra a B con wait_event = transactionid y bloqueado_por = {pid de A}. Al confirmar A, B recibe: «Copia NO disponible … está en estado 'Alquilada'».

Escenario	Pasos (3 ventanas psql)	Resultado verificado en PostgreSQL 16
2. REPEATABLE READ	B: BEGIN ISOLATION LEVEL REPEATABLE READ y consulta el catálogo. A alquila la copia 21. B vuelve a consultar e intenta alquilar la copia 21.	B sigue viendo el DVD disponible (instantánea fija) y al intentar alquilarlo recibe «could not serialize access due to concurrent update» (40001).
3. Deadlock	A bloquea el cliente 5; B bloquea la copia 18; A pide la copia 18; B pide el cliente 5.	«deadlock detected»: PostgreSQL aborta una sesión y la otra continúa.
4. NOWAIT (opcional)	A: SELECT … FOR UPDATE de la copia 22. B: el mismo SELECT con NOWAIT.	B falla de inmediato con «could not obtain lock on row» (55P03) en lugar de esperar.

9. Archivos Entregados y Orden de Ejecución

Archivo	Contenido
CineClub_Vintage_Documento_Entregable.docx	Este documento.
01_CineClub_DDL_DML.sql	Esquema, restricciones, índices, 5 triggers, 3 vistas y datos de prueba (12 películas, 27 copias, 12 clientes, 10 empleados, 14 alquileres y 20 detalles). Re-ejecutable.
02_CineClub_Validacion_Triggers.sql	24 casos controlados de éxito y rechazo con resumen PASA/FALLA; termina en ROLLBACK.
03_CineClub_Demo_Concurrencia.sql	Guion paso a paso para la sustentación con dos sesiones y un monitor de bloqueos.

Orden de ejecución en psql:

```
CREATE DATABASE cineclub_vintage;
\c cineclub_vintage
\encoding UTF8
\i C:/cineclub/01_CineClub_DDL_DML.sql
\i C:/cineclub/02_CineClub_Validacion_Triggers.sql
```

Codigo Pruebas:

Los datos de prueba usan fechas relativas a CURRENT_DATE, por lo que la mora y los bloqueos de clientes se reproducen igual el día de la sustentación.